

编译原理课程项目：Mandril v2.0

支持动态内存管理与函数抽象的极简语言设计

编译原理课程组

2026 年 4 月



福州大学
FUZHOU UNIVERSITY



- ① 简要介绍
- ② Mandrill 参考手册
- ③ 实验安排
- ④ 结束语



1 简介

2 Mandrill 参考手册

3 实验安排

4 结束语

项目背景与目标

- 核心目标：从零构建一个专用的解释器与编译器，理解程序设计语言的最基本原理。
- 设计理念：保持代码量尽可能少，但功能足以支持复杂程序（如冒泡排序、递归调用）。
- 版本演进：从 v1.0 的纯全局变量/整型系统，演进为支持动态数组、UTF-32 字符串及函数调用的 v2.0 系统。

核心语言特性

- 统一数据类型：全系统仅有 32 位整型（数值）与数组（本质为指针）。
- 类型推断：变量类型由其在源文件中首次出现的形式决定（例如 `a = 0` 推断为整数，`a[] = 10` 推断为数组）。
- 作用域控制：引入 `local` 与 `global` 关键字，支持基于 `{}` 的局部变量管理。
- 结构化控制流：支持 `while` 循环及 `break`、`continue` 语句。

内存模型与动态读入 (read)

- 堆内存管理：数组和字符串存储在堆区。
- 动态分配语义：针对数组的 read 操作始终涉及动态空间申请：
 - `a[] = read`：首次出现时声明数组并根据输入的字符串长度分配内存。
 - `a = read`：释放/丢弃旧存储空间，申请新空间存入新读入的字符串。
- 字符串内码：源文件使用 UTF-8，但编译器、解释器内部统一转换为 4 字节的 UTF-32 以简化处理。

过程抽象：函数定义

- 定义语法：使用 `func` 关键字。参数为值传递（数组参数传递指针地址）。
- 返回值类型：
 - `func name(a, b, c[])`：返回 32 位整数值。
 - `func[] name(a, b, c[])`：明确声明返回数组指针（堆地址）。
- 返回逻辑：通过 `return` 显式返回。若无返回语句，整数函数默认返回 0。
- 递归支持：函数可直接或间接调用自身，需正确处理调用栈和局部变量作用域。

代码示例：动态处理与控制流

```
1 # 演示数组动态读入与函数返回
2 func[] get_user_data() {
3     local str[];
4     str[] = read; # 首次出现：声明并动态申请空间
5     return str;
6 }
7 # 主程序
8 data = get_user_data();
9 i = 0;
10 while (data[i] != 0) {
11     if (data[i] == ' ') {
12         break; # 遇到空格跳出循环
13     }
14     put = data[i];
15     i = i + 1;
16 }
17 # 重新读入，丢弃旧空间
18 data = read;
19 write = data;
```


技术实现挑战

- 符号表设计：需处理嵌套作用域及函数名、局部/全局变量名的冲突检测。
- **VM** 堆管理：实现简单的 malloc/free 逻辑以支持 `a = read` 的空间更迭。
- 控制流跳转：在生成代码时，需维护 break/continue 的跳转目标栈。
- 内码转换：处理 UTF-8 变长字符到 UTF-32 定长字符的转换逻辑。



① 简介

② Mandrill 参考手册

词法
语法
语义

③ 实验安排

④ 结束语



① 简要介绍

② Mandrill 参考手册

词法
语法
语义

③ 实验安排

④ 结束语

词法定义 (Lexical Definitions)

词法分析器应将源代码识别为以下 Token 流:

① 关键字与字面量

- Keywords: if, else, while, read, put, write, get, func, global, local, return, break, continue
- Integer: $[0-9]^+$
- Identifier: $[a-zA-Z_][a-zA-Z0-9_]^*$
- CharConstant: ' (任何字符或转义字符 $\backslash n$, $\backslash \backslash$, $\backslash '$) '
- StringConstant: " (任何字符或转义字符序列) "

② 运算符与分隔符

- Operators: +, -, *, /, %, <, <=, >, >=, ==, !=, =
- Delimiters: (,), [,], , , ,, ;

③ 辅助项

- Comment: # 之后至行尾的所有内容 (忽略)
- Whitespace: 空格、制表符、换行符 (作为 Token 分隔符, 本身无语义)



① 简介

② Mandrill 参考手册

词法
语法
语义

③ 实验安排

④ 结束语

语法定义 (Grammar Definitions)

Listing 1: 顶层结构

```

1 Program      ::= { FunctionDef | Statement }
2
3 FunctionDef   ::= "func" [ "[" ] Identifier "(" [ ParameterList ] ")" Block
4
5 ParameterList ::= Identifier { "," Identifier }
```

语句 (续)

Listing 2: 语句

```

1 Statement      ::= AssignmentStmt
2                | IfStmt
3                | WhileStmt
4                | JumpStmt
5                | DeclarationStmt
6                | Block
7                | ";"
8
9 Block          ::= "{" { Statement } "}"
10
11 IfStmt         ::= "if" "(" Expression ")" Block "else" Block
12
13 WhileStmt      ::= "while" "(" Expression ")" Block
14
15 AssignmentStmt ::= LValue "=" Expression ";"
16                | Identifier "[" Expression "]" "=" Expression ";" /* 数组声明或初始化 */
    
```

语句

Listing 3: 语句 (续)

```

1  JumpStmt      ::= "break" ";"
2                  | "continue" ";"
3                  | "return" Expression ";"
4
5  DeclarationStmt ::= ( "local" | "global" ) Identifier [ "[" "]" ] ";"
    
```


表达式

Listing 4: 表达式

```

1  LValue          ::= Identifier [ "[" Expression "]" ]
2                    | "write"
3                    | "put"
4
5  Expression      ::= LogicalExp
6
7  LogicalExp      ::= RelationalExp { ("==" | "!=") RelationalExp }
8
9  RelationalExp   ::= AdditiveExp { ("<" | "<=" | ">" | ">=") AdditiveExp }
10
11 AdditiveExp     ::= MultiplicativeExp { ("+" | "-") MultiplicativeExp }
12
13 MultiplicativeExp ::= Primary { ("*" | "/" | "%") Primary }

```

表达式 (续)

Listing 5: 表达式 (续)

```

1 Primary      ::= Integer
2               | CharConstant
3               | StringConstant
4               | Identifier "(" [ ArgumentList ] ")" /* 函数调用 */
5               | LValue
6               | "read"
7               | "get"
8               | "(" Expression ")"
9
10 ArgumentList ::= Expression { "," Expression }
```



① 简要介绍

② Mandrill 参考手册

词法
语法
语义

③ 实验安排

④ 结束语

语义约束与补充说明——变量推断

全局变量或局部变量在**首次**声明或使用时根据上下文推断类型，具体规则如下：

- 有以下情况之一的，推断为整数变量：
 - 作为右值的，如 $a = b + 1$ ；中的 b ，推断为全局整数变量并赋予初始值 0。
 - $a = \text{expr};$ 、 $\text{local } a;$ 或 $\text{global } a;$ 且 expr 是结果为整数的表达式（字面量、变量、函数调用等）。
 - $a = \text{read};$ 且 a 未定义，则等价于先执行 $\text{global } a;$ 再执行 $a = \text{read};$ ，类型推断为整数变量。
- 有以下情况之一的，推断为数组变量：
 - $a = \text{expr};$ 、 $\text{local } a[];$ 或 $\text{global } a[];$ 且 expr 是结果为数组的表达式（函数调用、字符串字面值）。
 - $a[] = \text{expr};$ 且 expr 是结果为整数的表达式（字面值、变量、函数调用等）。
 - 如 $a[] = \text{read};$ 且 a 未定义，则等价于先执行 $\text{global } a[];$ 再执行 $a[] = \text{read};$ ，类型推断为数组变量¹。

¹特别地， $a = \text{"Hello"};$ 与 $a[] = \text{"Hello"};$ 均推断为数组变量 a ，并根据字符串长度动态分配内存存储 UTF-32 编码的 C 风格字符串内容。

语义约束与补充说明——输入输出

- 有以下情形之一的，识别为读取字符串，行为是读取连续非空白字符到指定变量中：
 - `a[] = read`: 当 `a` 未定义时，声明 `a` 为数组并根据输入字符串长度分配内存；当 `a` 已定义为数组时，释放旧空间并根据新输入申请新空间。
 - `a = read`: 当 `a` 已声明为数组时，释放旧空间并根据新输入申请新空间。
- 有以下情形之一的，识别为读取整数，行为是读取一个十进制整数值到指定变量中：
 - `a = read`: 当 `a` 未定义或已声明为整数变量时，`a` 为整数并将读入的整数值存入 `a`。

语义约束与补充说明（续）

- ③ 内存寻址逻辑：由于统一采用 UTF-32 编码和 32 位整数，所有数组索引 $a[i]$ 的底层地址计算逻辑为： $\text{Address} = \text{Base}(a) + (i \times 4)$ 字符串常量在数据段或堆区分配时，需确保每个字符占用 4 字节，并以 0x00000000 结尾。
- ④ 符号表设计建议
- Name: 标识符名称。
 - Type: INT 或 ARRAY_PTR。
 - Scope: GLOBAL 或 LOCAL（及嵌套层级索引）。
 - Offset/Address: 在栈帧中的偏移或全局区的地址。

Mandrill 示例代码

```
1 write = read + 1;
2 put = '\n';
```

输入

5

输出

6

Mandrill 示例代码 2

```
1  a = read;
2  b = read;
3  if (b > a) {
4      c = a;
5      a = b;
6      b = c;
7  }
8  while (a % b != 0) {
9      c = b;
10     b = a % b;
11     a = c;
12 }
13 write = b;
14 put = '\n';
```

输入

60 36

输出

12

Mandrill 示例代码 3

```
1 func strlen(s[]) {
2     local i; i = 0;
3     while (s[i] != 0) { i = i + 1; }
4     return i; }
5 func[] strcpy(a[], b[]) {
6     local la; local lb;
7     la = strlen(a); lb = strlen(b);
8     local c[]; c[] = la + lb + 1;
9     local i; i = 0;
10    while (i < la) {
11        c[i] = a[i]; i = i + 1; }
12    while (i < la + lb) {
13        c[i] = b[i - la]; i = i + 1; }
14    c[la + lb + 1] = 0; return c; }
15 a[] = "Hello_"; b[] = "World\n";
16 c[] = strcpy(a, b);
17 write = c;
```

输出

Hello World



1 简介

2 Mandrill 参考手册

3 实验安排

实验一：词法分析器（15%）

实验二：语法分析器（20%）

实验三：基于 ANTLR 的编译器语义分析器（15%）

实验四：虚拟机汇编代码输出与运行（30%）

报告撰写（20%）

4 结束语



1 简介

2 Mandrill 参考手册

3 实验安排

实验一：词法分析器（15%）

实验二：语法分析器（20%）

实验三：基于 ANTLR 的编译器语义分析器（15%）

实验四：虚拟机汇编代码输出与运行（30%）

报告撰写（20%）

4 结束语

词法分析器实现要求

- 识别所有合法 Token 并忽略注释和空白。
- 对于非法输入（如未闭合的字符串、无效字符等），应提供清晰的错误信息并安全退出（exit code 1）。
- 输出 Token 流供后续语法分析器使用，格式包含 Token 类型、值及位置信息（行号、列号）。
- 输出格式参考样例和代码



1 简介

2 Mandrill 参考手册

3 实验安排

实验一：词法分析器（15%）

实验二：语法分析器（20%）

实验三：基于 ANTLR 的编译器语义分析器（15%）

实验四：虚拟机汇编代码输出与运行（30%）

报告撰写（20%）

4 结束语

语法分析器实现要求

- 从 stdin 读入源代码，输出到 stdout。
- 输入源代码保证没有词法错误，但不保证没有语法错误。
- 如果输入的源代码没有语法错误，输出 PASS，否则输出 ERROR。



示例

```
1 | write = read + read;
```

```
1 | write read + read;
```

1 | PASS

1 | ERROR



1 简介

2 Mandrill 参考手册

3 实验安排

实验一：词法分析器（15%）

实验二：语法分析器（20%）

实验三：基于 ANTLR 的编译器语义分析器（15%）

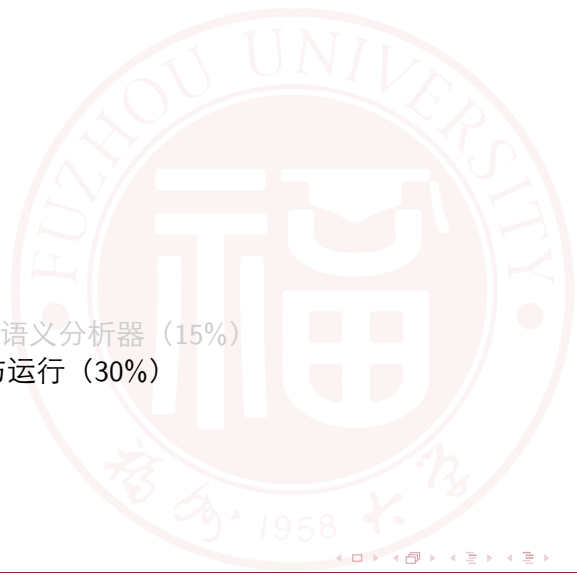
实验四：虚拟机汇编代码输出与运行（30%）

报告撰写（20%）

4 结束语

语法分析器实现要求

- 从 stdin 读入源代码，输出到 stdout。
- 输入源代码保证没有词法、语法错误，可能存在语义错误，比如对一个 `a=10;`，其中 `a` 为数组变量。
- 如果输入的源代码没有语义错误，输出 PASS，否则输出 ERROR。
- 建议通过 ANTLR 实现语义分析器，当然也可以通过其他方式实现。



1 简介

2 Mandrill 参考手册

3 实验安排

实验一：词法分析器（15%）

实验二：语法分析器（20%）

实验三：基于 ANTLR 的编译器语义分析器（15%）

实验四：虚拟机汇编代码输出与运行（30%）

报告撰写（20%）

4 结束语

虚拟机汇编代码输出与运行要求

- 从 `stdin` 读入源代码，输出字节码到 `stdout`。
- 部署在评测机上的虚拟机程序将从 `stdin` 读入字节码并执行，输出结果到 `stdout`。
- 输入的源代码保证不存在语法和词法错误。
- 保证源代码在给定输入的情况下不会发生死循环。

Mandril 虚拟机

- Mandril 虚拟机采用类似 Java 最初版本的虚拟机架构，即虚拟机主要维护一个栈，用于存储操作数。
- 此外，虚拟机还会维护两个寄存器分别是程序计数器（Program Counter，PC）和栈指针（Stack Pointer，SP）²。
- 程序计数器是一个 32 位整数，指向代码存储区中下一轮执行指令的第一字节地址，从 0x00000000 开始，每条指令占 8 个字节，正常指令执行后 PC 自动加 8。跳转和分支求值指令会修改 PC 的值。
- 当 PC 恰好等于 0xFFFFFFFF 时，虚拟机退出并返回 0。即程序执行的最后一条指令强制为 jump 0xFFFFFFFF。

²指向第一个局部变量的地址

虚拟机汇编——数据存取指令（入栈）

Mandrill 虚拟机的指令一般由‘命令+操作对象’构成，其中命令为固定单词，操作对象为立即数、运算符类型以及内存地址构成。

接下来将介绍每一个指令。

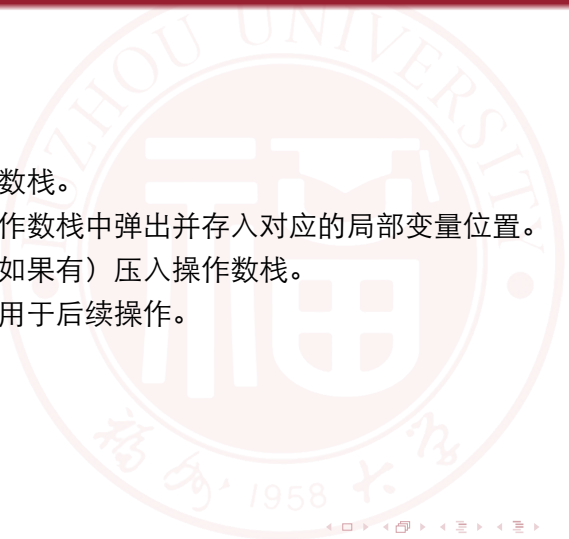
- 全局数据加载指令：用于从数据存储区对应位置读取数据并存入操作数栈。
格式：dload x, x 为变量编号，从 0 开始。
- 局部数据加载指令：用于从数据存储区对应位置读取数据并存入操作数栈。
格式：dllload x, x 为局部变量编号，从 0 开始，是相对于栈指针的偏移量。
- 绝对数据加载指令：从栈顶获取地址并从对应的内存地址中读取数据并存入操作数栈，可用于访问数组元素。
格式：daload x, x 为无用值填充，仅为保证指令长度。
- 数据存储指令：用于将数据（立即数）压入操作数栈。
格式：dstore x, x 为立即数。

虚拟机汇编——数据存取指令（出栈）

- 数据写入指令：用于将操作数栈顶的元素写入对应存储位置并将其弹出。
格式：dwrite x, x 为目标变量编码。
- 全局数据写入指令：用于将操作数栈顶的元素写入对应存储位置并将其弹出。
格式：dwrite x, x 为目标变量编码。
- 局部数据写入指令：用于将操作数栈顶的元素写入对应存储位置并将其弹出。
格式：dlwrite x, x 为目标变量编码。
- 绝对数据写入指令：需要从栈顶获取两个操作数，栈顶元素为地址，第二个元素为要写入的值，将值写入对应的内存地址并将它们弹出。
格式：dawrite x, x 为无用值填充，仅为保证指令长度。

函数调用的实现示例

- ① 调用者将函数参数依次压入操作数栈。
- ② 被调用者将函数参数一次性从操作数栈中弹出并存入对应的局部变量位置。
- ③ 被调用者执行完毕后将返回值（如果有）压入操作数栈。
- ④ 调用者从操作数栈中获取返回值用于后续操作。



虚拟机汇编——求值

- 求值指令：根据立即数 x 进行对应操作。
如果 x 对应二元运算，则取出操作数栈顶两个元素进行对应的计算后将结果存回操作数栈顶。
如果 x 对应条件跳转运算，则取出三个操作数，第一个元素（原栈顶）为表达式的结果。若表达式结果不为零，将 PC 设置为第二个元素（次栈顶，THEN 分支的地址），否则设置为第三个元素（再次栈顶，ELSE 分支的地址）。³
格式：eval x ， x 为立即数，运算类型与对应的值参考后面字节码介绍。

³编译器在生成代码时需要预先计算出 THEN 分支和 ELSE 分支的地址并将它们作为立即数通过 `dstore x` 压入栈中。

虚拟机汇编——跳转

- 直接跳转指令：该指令将直接把 PC 设置为 x 。
格式：jump x 。
- 链接跳转指令：记录当前的栈寄存器和 PC⁴，并根据栈顶元素申请内存，把栈寄存器设置为新的内存地址。然后将直接把 PC 设置为 x 。
格式：jal x 。
- 返回指令：从虚拟机控制的存储区中获取原来的栈寄存器和 PC 值并设置回去。⁵
格式：ret x ， x 忽略。
- 无操作指令：不执行任何操作。
格式：nop 0，后面的操作对象可以是任意整数，均应当被忽略。

⁴保存在由虚拟机管理的空间，对用户来说是透明的

⁵编译器生成代码时应当将生成 dstore、dload 等指令手动将函数返回值压入栈中，以供调用者获取。

虚拟机汇编——内存分配

- 内存分配指令：从操作数栈中获取一个元素作为需要分配的内存大小（单位为字节），申请对应大小的内存并将起始地址压入操作数栈。
格式：malloc x，x 为无用值填充，仅为保证指令长度。⁶

⁶对于字符串字面值而言，需要生成申请内存空间的指令以及逐字符复制的指令。此外，还需要额外处理存储内存地址的计算，如 $a[i] \rightarrow a + i * 4$ 。

虚拟机汇编——输入输出指令

- 整数输入指令：操作数无用。
格式：geti x，将从 stdin 读入一个十进制整数的数据存入栈顶。
- 字符输入指令：操作数无用。
格式：getc x，将从 stdin 读入一个字符并把它对应的 ASCII 值存入栈顶。
- 字符串输入指令：操作数无用。
格式：gets，将从 stdin 读入一个连续非空字符构成的字符串并把地址存入栈顶。
- 整数输出指令：操作数无用。
格式：puti x，将栈顶元素弹出并以十进制的形式输出到 stdout 中。
- 字符输出指令：操作数无用。
格式：putc x，将栈顶元素弹出转换为 UTF-8 字符输出到 stdout 中，否则什么都不做。
- 字符串输出指令：操作数无用。
格式：puts x，将栈顶元素弹出并判断它是否为合法的内存地址，若合法则从该地址开始按照 C 风格字符串要求转换为 UTF-8 编码后输出到 stdout 中，否则什么都不做。

EVAL 指令操作数

eval 指令操作数:

- 加法: 0x00010001
- 减法: 0x00010002
- 乘法: 0x00010003
- 整除: 0x00010004
- 求余: 0x00010005
- 大于: 0x00010006
- 小于: 0x00010007
- 大等于: 0x00010008
- 小等于: 0x00010009
- 等于: 0x0001000A
- 不等于: 0x0001000B
- 条件跳转: 0x0001000C



1 简介

2 Mandrill 参考手册

3 实验安排

实验一：词法分析器（15%）

实验二：语法分析器（20%）

实验三：基于 ANTLR 的编译器语义分析器（15%）

实验四：虚拟机汇编代码输出与运行（30%）

报告撰写（20%）

4 结束语

实验报告要求

撰写一个 Mandrill 语言编译器完整实践过程的报告，包括但不限于

- 技术路线：词法分析器设计，语法及语法分析器设计（用 LL(1)、LR 还是 LALR 算法等）；
- 实现的优化功能：常量压缩、短路表达式（自动检测逻辑与、或并在确定结果的情况下提前返回结果）等；
- 实现过程的心得体会；
- 致谢；



- ① 简介
- ② Mandrill 参考手册
- ③ 实验安排
- ④ 结束语

祝愿大家能够借本课程实践项目的机会巩固理论知识，提高编程水平，为未来的学习、研究、工作打下坚实的基础。